

Van and Bus Booking System

Project 2 — Team Task Division & AI Work Guide

CSX4107 (542) | Web Application Development | September 2026

Item	Details
Team members	Li Hout Van; Ye Htet Aung; Zaw Zaw Naing
Technology	Next.js + React, MongoDB + Mongoose, RESTful APIs, Session or JWT authentication
Core modules	Vehicles, Trips, Bookings, Seat Availability, Authentication, Responsive UI

Purpose of this document

This guide divides the project into three connected ownership areas. It follows the submitted proposal and gives each member clear responsibilities, deliverables, dependencies, and a focused AI prompt.

Priority: Complete the three required CRUD models and booking integrity before optional features such as the Driver view.

1. Project Scope and Shared Goal

The Van and Bus Booking System is a full-stack web application for managing vehicles, scheduled trips, seats, and bookings. It creates one source of truth for transport data and reduces problems caused by manual booking workflows.

Project goal: Build a secure, responsive web application that digitizes the core workflow of publishing transport trips and reserving seats.

Required features from the proposal:

- Vehicle management: create, view, edit, and deactivate vehicles.
- Trip management: schedule, view, edit, and cancel trips.
- Live seat availability for scheduled trips.
- Customer booking creation, review, and cancellation before departure.
- Duplicate-booking prevention for the same trip and seat.
- Validation, status management, and clear REST API responses.
- Authentication and role-based access for administrators and customers.
- Responsive desktop and mobile interface.

Out of scope: online payments, real-time GPS tracking, native mobile apps, multi-company tenancy, and automated SMS/email integration.

2. Main Task Division

Member	Main ownership	Specific responsibilities	Deliverable
Li Hout Van	Vehicle & Trip Management	Vehicle CRUD; Trip CRUD; admin pages; capacity/status; scheduling; cancellation; validation.	Admin transport module
Ye Htet Aung	Booking & Seat Availability	Booking CRUD; seat layout; availability API; duplicate-seat prevention; confirmation; cancellation.	Booking and seat module
Zaw Zaw Naing	Authentication, Customer UI & Integration	Registration/login; roles; protected routes; trip search; customer pages; responsive UI; API integration.	Customer UI and authentication

Ownership rule: Each member implements, tests, and explains their module. Integration and final testing are shared team responsibilities.

3. Member 1 — Li Hout Van

Role: Vehicle and Trip Management Lead

Main objective: Build the administrator's transport-management functionality and provide reliable vehicle/trip data for booking.

Tasks

- Create Vehicle schema: plateNumber, type, capacity, status.
- Implement Vehicle endpoints: GET/POST /api/vehicles and GET/PATCH/DELETE /api/vehicles/:id.
- Support vehicle creation, viewing, editing, and deactivation.
- Validate capacity > 0 and unique plateNumber.
- Create Trip schema: vehicleId, origin, destination, departureTime, fare, status.
- Implement Trip endpoints: GET/POST /api/trips and GET/PATCH/DELETE /api/trips/:id.
- Support trip creation, viewing, editing, and cancellation.
- Ensure trips reference active vehicles.

- Validate departure time earlier than arrival time.
- Build admin vehicle and trip pages.
- Provide trip data for customer search and booking.

Expected deliverables

- Vehicle model and REST API
- Trip model and REST API
- Admin vehicle page
- Admin trip page
- Validation and error handling
- Seed vehicles and trips

Dependencies

- Agree with Member 2 on Trip/Vehicle fields used by bookings.
- Provide stable endpoint names and JSON formats.
- Coordinate with Member 3 for customer trip display.

Focused AI prompt

You are helping me build my assigned module for a Next.js + MongoDB + Mongoose Van and Bus Booking System. My role: Vehicle and Trip Management. Requirements: - Vehicle: plateNumber, type, capacity, status. - Trip: vehicleId, origin, destination, departureTime, fare, status. - Vehicle operations: create, read, update, deactivate. - Trip operations: create, read, update, cancel. - Capacity must be greater than zero and plateNumber must be unique. - Trips must reference active vehicles. - Departure time must be earlier than arrival time. - Use Next.js Route Handlers and validated JSON responses. Rules: 1. Give only the code or steps needed for my specific task. 2. Keep explanations short. 3. Do not add payment, GPS, SMS, or unrelated features. 4. Reuse code I provide; do not rewrite unrelated files. 5. Ask one concise question if required information is missing. 6. State the exact file path for every code change. First ask whether I am working on a schema, API route, admin page, validation, or test.

4. Member 2 — Ye Htet Aung

Role: Booking and Seat Availability Lead

Main objective: Build the reservation workflow and ensure seats cannot be double-booked or assigned incorrectly.

Tasks

- Create Booking schema: tripId, userId, seatNumbers, passengerName, status.
- Implement Booking endpoints: GET/POST /api/bookings and GET/PATCH/DELETE /api/bookings/:id.
- Allow customers to create, view, and cancel their own bookings before departure.
- Allow administrators to view bookings and update booking status when required.
- Implement GET /api/trips/:id/availability.
- Return booked seats and remaining capacity.
- Build the available-seat layout for a selected trip.
- Reject seats already reserved for the same trip.
- Reject seat numbers outside vehicle capacity.
- Ensure cancelled bookings release seats.
- Prevent bookings for departed or completed trips.
- Provide in-application booking confirmation.

Expected deliverables

- Booking model and REST API
- Seat availability endpoint
- Seat-selection interface
- Booking form and confirmation
- Cancellation/status handling
- Duplicate-seat prevention tests
- Seed bookings

Dependencies

- Use Member 1's Trip and Vehicle references.
- Use Member 3's authenticated user information and role rules.
- Agree on booked-seat representation and availability response.

Focused AI prompt

You are helping me build my assigned module for a Next.js + MongoDB + Mongoose Van and Bus Booking System. My role: Booking and Seat Availability. Requirements: - Booking: tripId, userId, seatNumbers, passengerName, status. - Operations: create, read, update status, cancel. - Endpoint: GET /api/trips/:id/availability. - Customers can book, review, and cancel their own bookings before departure. - Show available seats and remaining capacity. - A seat cannot be confirmed twice for the same trip. - Seats must be within vehicle capacity. - Cancelled bookings release seats. - Departed/completed trips reject new bookings. - Use server-side validation. Rules: 1. Give only the code or steps needed for my specific task. 2. Keep explanations short and focus on booking correctness. 3. Consider duplicate-seat protection and race conditions. 4. Do not add payment, GPS, SMS, or unrelated features. 5. Reuse code I provide and avoid unrelated rewrites. 6. State exact file paths. 7. Ask for the current schema/API instead of guessing. First ask whether I am implementing the schema, availability API, booking API, seat UI, cancellation, or tests.

5. Member 3 — Zaw Zaw Naing

Role: Authentication, Customer UI, and Integration Lead

Main objective: Build the user-facing experience, protect access, and connect all modules into one responsive application.

Tasks

- Create User model: name, email, passwordHash, role.
- Implement registration and login.
- Use session-based or JWT-based authentication.
- Support administrator and customer roles.
- Protect administrator actions with role-based access.
- Ensure customers manage only their own bookings.
- Build customer trip-browsing and trip-details pages.
- Implement search/filtering by route, date, and availability.
- Connect customer UI to the Trip API.
- Connect booking pages to Booking and Availability APIs.
- Build reusable components, navigation, forms, loading states, and errors.
- Ensure responsive desktop/mobile layouts.
- Coordinate final integration.

Expected deliverables

- User model and authentication flow
- Login and registration pages
- Role-based route protection
- Customer trip-list and details pages
- Responsive UI integration
- Reusable components and shared layout
- Integrated navigation and error handling

Dependencies

- Use the agreed APIs from Members 1 and 2.
- Do not change shared response formats without agreement.
- Confirm authenticated user ID and role format before connecting booking ownership.

Focused AI prompt

You are helping me build my assigned module for a Next.js + MongoDB + Mongoose Van and Bus Booking System. My role: Authentication, Customer UI, and Integration. Requirements: - User: name, email, passwordHash, role. - Roles: administrator and customer. - Use session-based or JWT-based authentication. - Admin actions must be protected. - Customers can manage only their own bookings. - Customers browse/filter trips, inspect details, select seats, and review bookings. - UI must be responsive on desktop and mobile. - Use reusable React components and clear loading/error states. Rules: 1. Give only the code or steps needed for my specific task. 2. Keep explanations short. 3. Do not invent API endpoints. 4. Reuse existing code and avoid unrelated rewrites. 5. Do not add payment, GPS, SMS, or unrelated features. 6. State exact file paths. 7. Briefly explain security-sensitive decisions. 8. Ask one concise question if the API contract or auth method is missing. First ask whether I am working on the User model, registration, login, role protection, customer pages, responsive UI, or integration.

6. Shared API and Data Contract

The proposal defines these REST resources. All members should agree on field names and response formats before implementing dependent features.

Resource	Endpoint	Purpose
Vehicles	GET/POST /api/vehicles	List vehicles or create a vehicle
Vehicles	GET/PATCH/DELETE /api/vehicles/:id	View, update, or deactivate a vehicle
Trips	GET/POST /api/trips	Search/list trips or create a trip
Trips	GET/PATCH/DELETE /api/trips/:id	View, update, or cancel a trip
Bookings	GET/POST /api/bookings	List permitted bookings or create a booking
Bookings	GET/PATCH/DELETE /api/bookings/:id	View, update status, or cancel a booking
Availability	GET /api/trips/:id/availability	Return booked seats and remaining capacity

Recommended response shape

```
Success: {"success": true, "data": {...}, "message": "Request successful"}
Error: {"success": false, "error": "Validation message"}
```

Shared rules

- Use identical field names across schemas, APIs, and forms.
- Do not duplicate models in multiple files.
- Use environment variables for connection strings and secrets.
- Never store plain-text passwords.
- Validate important rules on the server.
- Use clear HTTP status codes and messages.
- Document endpoint changes before other members depend on them.

7. Full-System Workflow

- 1 Admin creates a vehicle with capacity and active status.
- 2 Admin creates a trip with vehicle, route, schedule, fare, and status.
- 3 Customer searches for a trip by route and date.
- 4 Customer opens trip details.
- 5 System displays available seats.
- 6 Customer selects a seat and submits passenger details.
- 7 Booking API validates authentication, trip status, capacity, and availability.
- 8 Booking is saved only if the selected seat is still free.
- 9 Customer views booking confirmation.
- 10 Admin monitors booking and trip status.

Integration checkpoints

- Trip API returns trip and vehicle capacity.
- Availability API returns booked seats and remaining capacity.
- Customer page displays real trip data.
- Customer creates a booking through the real API.
- Booked seat becomes unavailable.
- Cancelled booking releases the seat.

- Admin and customer permissions work correctly.

8. Five-Week Work Plan

Week	Member 1	Member 2	Member 3	Team output
1 — Requirements, wireframes, schemas	Vehicle/Trip schema and admin wireframes.	Booking schema, seat logic, workflow.	User schema, auth flow, customer wireframes.	Approved scope, UI sketches, MongoDB plan.
2 — Vehicle and Trip CRUD	Build/test APIs and admin pages.	Prepare booking references.	Auth foundation and trip UI.	Admin pages and tested endpoints.
3 — Booking and availability	Support capacity/status integration.	Booking flow, seats, availability, duplicate prevention.	Connect customer UI to APIs.	Booking flow with duplicate prevention.
4 — Auth, validation, responsive UI	Check admin and trip validation.	Test cancellation and edge cases.	Complete auth, roles, responsive UI, integration.	Role protection and mobile refinement.
5 — Testing, docs, presentation	Test admin module and demo.	Test booking module and demo.	Test full UI/auth and integrated demo.	Seed data, evidence, README, presentation.

9. AI Usage Rules for All Members

Use the assigned prompt first. To reduce token usage and prevent inconsistent code:

- Ask for one feature, one file, or one error at a time.
- Paste only relevant code, not the whole project.
- Always provide the exact file path.
- Ask for a small patch instead of a full project rewrite.
- Do not add libraries or architecture changes without team agreement.
- Run and test generated code locally before committing.
- Review generated code against the proposal.
- Never paste passwords, API keys, database credentials, or private secrets.

Efficient follow-up template

Task: [one task]
 File: [exact path]
 Current code: [relevant code only]
 Error/expected behavior: [details]
 Request: Give the smallest safe fix, changed code only, and 2–4 short notes. Do not rewrite unrelated files.

10. Git, Integration, and Testing Rules

Suggested branches

- feature/vehicle-trip-management
- feature/booking-seat-availability
- feature/auth-customer-ui
- integration/testing

Before merging

- Pull the latest shared branch.
- Run the application locally.

- Test the changed feature and one related workflow.
- Confirm schemas and API responses match the shared contract.
- Use a clear commit message.
- Tell the team which files, endpoints, or fields changed.

Minimum test checklist

Area	Required check	Owner
Vehicle	Create, edit, deactivate, unique plate, capacity > 0	Member 1
Trip	Active vehicle, valid times, edit, cancel	Member 1
Booking	Create, view, cancel, status update	Member 2
Seats	Display availability, reject duplicate/out-of-range seat	Member 2
Authentication	Register, login, password hashing, invalid login	Member 3
Authorization	Admin protection and own-booking restriction	Member 3 + all
Responsive UI	Desktop and mobile layouts work	Member 3 + all
Full workflow	Admin creates trip → customer books → seat unavailable	All

11. Final Acceptance Checklist

Before the final demonstration, confirm that:

- Vehicle CRUD works; plate numbers are unique; capacity/status are correct.
- Trip CRUD works; trips reference active vehicles; cancellation/status work.
- Customers search trips and view available seats.
- Customers create, review, and cancel their own bookings before departure.
- Duplicate booking of the same trip and seat is rejected.
- Cancelled bookings release seats.
- Departed/completed trips reject new bookings.
- Administrator and customer roles are protected.
- Passwords are hashed and secrets are not exposed.
- REST APIs return validated JSON responses.
- MongoDB relationships and constraints work.
- Responsive pages work on desktop and mobile.
- Seed data exists for demonstration.
- README includes setup, API summary, data model, and test evidence.

Final team rule

This is one project, not three separate mini-projects. Each member owns a module, but all code must follow the same data model, API contract, naming conventions, and workflow.